

Bases de Datos 2

Final – 09/06/2023

- 1) Explique brevemente qué es JPA y para qué fue creado.

JPA (Java Persistence API) es un estándar Java que define un conjunto de interfaces y anotaciones para trabajar con ORM. Hibernate, EclipseLink y otros son implementaciones de JPA. Con JPA, se definen entidades: clases Java simples (POJOs) con getters, setters y un constructor vacío, marcadas con `@Entity`. JPA por convención persiste todos los atributos de una entidad a menos que se anoten con `@Transient`. Se configura la persistencia con un archivo `persistence.xml` o anotaciones, siguiendo principalmente el paradigma “convención sobre configuración” (por ejemplo, una propiedad `name` se mapea a columna `NAME` por defecto).

- 2) ¿Cuáles son las reglas para persistencia de POJOs en Hibernate?

- Implementar un constructor vacío.
- Las clases a persistir deben tener un atributo id..
- Setters y Getters para campos persistentes.
- Clases no finales: una clase final no puede tener subclases. Esto es porque hibernate utiliza el patrón proxy para representar los objetos de la BD, y para poder implementar el proxy, se debe poder crear una subclase de mi clase para luego hacer que el proxy entienda todos los mensajes que entiende mi clase.

- 3) ¿Cuál es la diferencia principal entre DAO y el patrón Repository?

La diferencia principal entre DAO y el patrón Repository radica en el alcance de sus responsabilidades: mientras que el DAO concentra todas las operaciones sobre la base de datos (guardar, actualizar, eliminar y obtener), lo que suele trasladar parte del comportamiento de negocio a la capa de persistencia y debilitar el modelo, el Repository se limita a la obtención (fetch) de objetos y delega las operaciones de guardado y actualización a mecanismos automáticos como la persistencia por alcance, manteniendo así un modelo de dominio más limpio y menos dependiente de la capa de acceso a datos.

- 4) Defina persistencia por alcance.

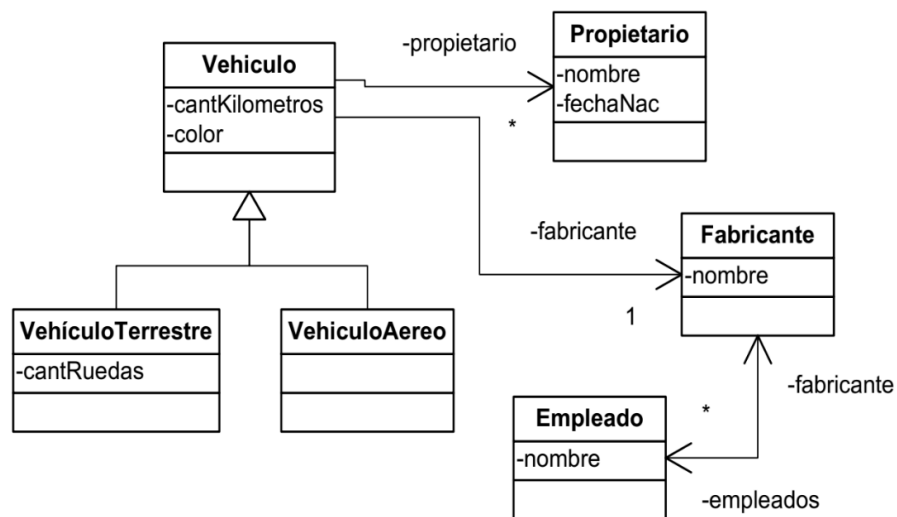
“Todo objeto alcanzable desde un objeto persistente debe ser también persistente”. Esto significa que si un objeto A ya está guardado y referencia a otro objeto B, entonces B también se almacenará automáticamente cuando se persista A. El ORM se encarga de seguir las referencias y guardar el grafo completo de objetos relacionados.

- 5) Explique brevemente como es el control de concurrencia optimista basado en versiones que utiliza Hibernate.

Esta estrategia supone que no existirán conflictos de ediciones concurrentes de la misma información, por diferentes usuarios. A partir de esta suposición se permite el acceso a los usuarios a todos los objetos del repositorio. JPA permite poner `@Version` en una entidad, de modo que Hibernate detecta actualizaciones

concurrentes e impide sobrescribir cambios sin saberlo, lanzando excepciones de locking optimista.

- 6) Considerando Java Data Objects (JDO). Comente alguno de los posibles estados en los que se encuentra una instancia al ser persistida con esta tecnología.
- Transient
 - Attached
 - Persisted
 - Deleted
- 7) ¿Qué significa que la persistencia debería ser ortogonal?
Cualquier objeto debería poder ser persistido, independientemente del tipo.
- 8) Dado el siguiente modelo escriba en HQL una consulta que obtenga los Vehículos aéreos con menos de 400.000 km, de propietarios nacidos en 1985 y fabricante "Airbus".



```
FROM VehiculoAereo va
WHERE va.cantKilometros < 400000
AND va.propietario.fechaNac > 1984-12-31
AND va.propietario.fechaNac < 1986-01-01
AND va.fabricante.nombre = "Airbus"
```

- 9) Dado el siguiente elemento de mapeo de clases
- ```
<class name="VIPUser" table="tableName" discriminator-value="2" lazy="true"
abstract="false" ... />
```
- Describe para qué se utiliza "discriminator-value"
- El discriminator-value se utiliza porque se está utilizando herencia, y para manejarla en el contexto relacional se está utilizando la estrategia SINGLE\_TABLE.
- discriminator-value funciona para diferenciar a que clase pertenece cada tupla dentro de la tabla tableName.

- 10) Usualmente las soluciones de persistencia utilizan el patrón de diseño Proxy en su implementación. Explique cuáles son los beneficios que brinda su aplicación y en qué circunstancias se utiliza.

El uso de proxies trae la ventaja del Lazy Loading, que se relaciona con el uso de FetchType.LAZY en relaciones entre objetos. Como beneficios ofrece transparencia, desde el punto de vista del programador se tiene el objeto completo junto con los relacionados; mayor rendimiento y eficiencia en memoria, ya que no carga la memoria con objetos no deseados.

## Bases de Batos 2

Final - 09/08/2024

- 1) Defina persistencia por alcance.

“Todo objeto alcanzable desde un objeto persistente debe ser también persistente”.

Esto significa que si un objeto A ya está guardado y referencia a otro objeto B, entonces B también se almacenará automáticamente cuando se persista A. El ORM se encarga de seguir las referencias y guardar el grafo completo de objetos relacionados.

- 2) ¿Qué limitaciones de las BBDD relacionales buscan superar las BBDD orientadas a documentos? Mencione al menos dos.

Buscan brindar flexibilidad, escalabilidad, disponibilidad y la capacidad de ser distribuidos.

- 3) Mencione diferencias entre JDO y Hibernate. Compare los principales elementos presentes en cada uno de ellos.

-

- 4) Considerando Java Data Objects (JDO). Comente alguno de los posibles estados en los que se encuentra una instancia al ser persistida con esta tecnología.

- Transient
- Attached
- Persisted
- Deleted

- 5) Explique brevemente como es el control de concurrencia optimista basado en versiones que utiliza Hibernate.

Esta estrategia supone que no existirán conflictos de ediciones concurrentes de la misma información, por diferentes usuarios. A partir de esta suposición se permite el acceso a los usuarios a todos los objetos del repositorio. JPA permite poner @Version en una entidad, de modo que Hibernate detecta actualizaciones concurrentes e impide sobrescribir cambios sin saberlo, lanzando excepciones de locking optimista.

- 6) ¿Para qué sirve la propiedad FetchType en Hibernate? Describa un caso de uso para cada valor que puede tomar.

Describe la forma en la que se solicitará un objeto junto a los que tiene relacionados cuando se haga un fetch a la base de datos. FetchType.LAZY hace que el objeto relacionado no se traiga junto al solicitado, por el otro lado, FetchType.EAGER trae el objeto relacionado junto al solicitado.

Si tomamos en cuenta el trabajo realizado, para el caso de una ruta que posee muchas paradas, el FetchType indicado es el EAGER, ya que es información importante para el usuario saber cuáles son las paradas que tiene una ruta. En cambio, si tomamos el caso de un usuario, las compras que haya realizado no son tan importantes, por lo que elegir FetchType.LAZY será correcto.

- 7) Usualmente las soluciones de persistencia utilizan el patrón de diseño Proxy en su implementación. Explique cuáles son los beneficios que brinda su aplicación y en qué circunstancias se utiliza.

El uso de proxies trae la ventaja del Lazy Loading, que se relaciona con el uso de FetchType.LAZY en relaciones entre objetos. Como beneficio ofrece transparencia, desde el punto de vista del programador se tiene el objeto completo junto con los relacionados; mayor rendimiento y eficiencia en memoria, ya que no carga la memoria con objetos no deseados.

- 8) Explique el principio de ortogonalidad en ORMs.

Cualquier objeto debería poder ser persistido, independientemente del tipo. Aunque hay objetos que no tienen sentido que sean persistentes, como objetos temporales para calcular otras cosas.

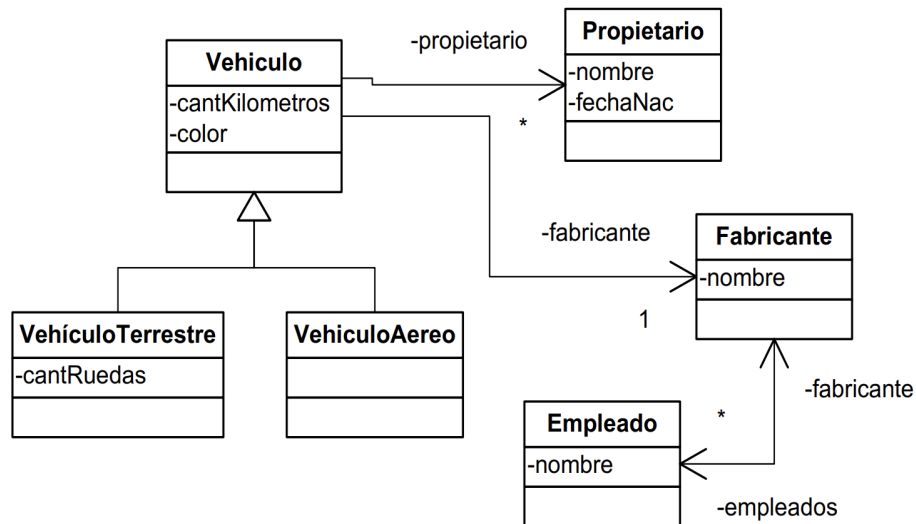
- 9) Explique las diferencias principales entre HQL y Criteria  
HQL

- Basada en SQL. Sigue siendo un string.
- Lenguaje intermedio entre objetos y SQL.
- No utiliza Select, ya que retorna objetos ya mapeados.
- Además permite retornar:
  - Arrays de objetos
  - Crear objetos nuevos
- Abstracción media, velocidad media, libertad media.
- FROM persona p WHERE p.nombre like "%Matias"

Criteria

- Basada en HQL.
- Query construida por mensajes, como en objetos.
- Mayor abstracción, peor velocidad, peor libertad.
- session.createCriteria(Persona.class).add(Expression.like("nombre", "%Matias %"))

- 10) Dado el siguiente modelo escriba en HQL una consulta que obtenga los Vehículos aéreos de fabricante "Cessna" y propietarios nacidos antes de 1980.



FROM VehiculoAereo va  
 WHERE va.fabricante.nombre = "Cessna"  
 AND va.propietario.fechaNac < 1980-01-01

## Bases de Batos 2

### Final - 13/12/2024

- 1) Mencione diferencias entre Java Data Objects (JDO) y Hibernate. Compare los principales elementos presentes en cada uno de ellos.  
 -
- 2) ¿Cuáles son las reglas para persistencia de POJOs en Hibernate? Indique cuáles son obligatorias y cuáles opcionales.
  - Implementar un constructor vacío.
  - Las clases a persistir deben tener un atributo id..
  - Setters y Getters para campos persistentes.
  - Clases no finales: una clase final no puede tener subclases. Esto es porque hibernate utiliza el patrón proxy para representar los objetos de la BD, y para poder implementar el proxy, se debe poder crear una subclase de mi clase para luego hacer que el proxy entienda todos los mensajes que entiende mi clase.
- 3) Defina persistencia por alcance.  
 "Todo objeto alcanzable desde un objeto persistente debe ser también persistente".  
 Esto significa que si un objeto A ya está guardado y referencia a otro objeto B, entonces B también se almacenará automáticamente cuando se persista A. El ORM se encarga de seguir las referencias y guardar el grafo completo de objetos relacionados.
- 4) Mencione características de los DTOs y por qué son necesarios. ¿En qué casos los utilizaría?

Son objetos no persistentes que sirven para copiar objetos persistentes y poder utilizarlos fuera de una transacción. Pueden ser diseñados de forma igual o distinta (con menos campos) que el objeto original, no necesitan ser iguales se los puede formatear a conveniencia. Se suelen utilizar en el desarrollo de APIRest's, para lógica que conecta el backend con el frontend

- 5) Al mapear una jerarquía de clase, ¿Cuáles son las ventajas y desventajas de la estrategia "single table" (una tabla para toda la jerarquía)?

Ventajas:

- Al modificar la clase padre, solo debe modificarse la tabla creada.
- Toda la información de las subclases se encuentra junta, por lo que se ahorran JOINS al momento de hacer un fetch de datos. También aplica al querer hacer un fetch de todos los objetos de la clase padre.

Desventajas:

- Al mezclar objetos de distintos tipos en una misma tabla, muchas columnas quedan vacías.
- Se dificulta el uso de variables sintácticamente parecidas pero semánticamente diferentes, ya sea que difieren por su uso o rango de valores.
- Se agrega una columna extra para diferenciar de qué clase estamos hablando.

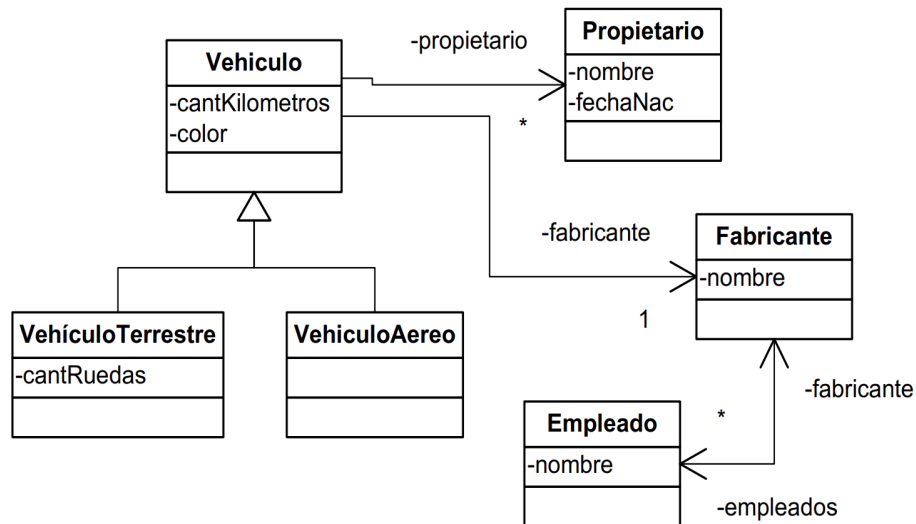
- 6) Explique el motivo por el cual se utiliza el patrón de diseño Proxy en soluciones de persistencia Objeto-Relacional.

El uso de proxies se utiliza principalmente por el Lazy Loading, que se relaciona con el uso de FetchType.LAZY en relaciones entre objetos. Como beneficios ofrece transparencia, desde el punto de vista del programador se tiene el objeto completo junto con los relacionados; mayor rendimiento y eficiencia en memoria, ya que no carga la memoria con objetos no deseados.

- 7) Explique cómo funciona la estrategia de recuperación EAGER en Hibernate, y en qué caso la utilizaría.

Cuando un objeto que tiene como atributo otro objeto persistente anotado como FetchType.EAGER, el objeto relacionado será devuelto de la base de datos al hacer un fetch del objeto original. Un caso sería en el de las rutas con sus paradas en el TP. Como las paradas son un dato importante a saber sobre las rutas, es importante que su información sea traída al hacer un fetch de una ruta.

- 8) Dado el siguiente modelo escriba en HQL una consulta que obtenga los Vehículos aéreos de fabricante "Cessna" y propietarios nacidos después de 1970.



FROM VehiculoAereo va  
 WHERE va.fabricante.nombre = "Cessna"  
 AND va.propietario.fechaNac > 1970-31-12

9) ¿Qué son los principios BASE de las bases de datos NoSQL?

Los principios BASE son para las bases de datos no relacionales lo que los principios ACID son para las bases de datos relacionales. BASE refiere a básicamente disponible (BA), estado flexible (S) y eventualmente consistente (E): "Una aplicación trabaja básicamente todo el tiempo (BA), no requiere ser consistente todo el tiempo (S), pero eventualmente estará en un estado conocido (E)". Proveen disponibilidad, es optimista, y permite un estado flexible(no consistente todo el tiempo) en la base de datos.

10) ¿Qué limitaciones de las BBDD relacionales buscan superar las BBDD orientadas a documentos? Mencione al menos dos.

Buscan brindar flexibilidad, escalabilidad, disponibilidad y la capacidad de ser distribuidos.